Field Hours

CSCE 310 Database Systems

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 50 minutes

Tables and keys

A table is rows of the same columns. A key is a column, or several, that identifies a row. A primary key does not repeat and is not null. A foreign key is a value that must already exist as a key in another table. The foreign key is how two tables refer to each other without copying a whole row.

If you paste a customer's address onto every order, a move requires many updates, and you will miss one. Store the address once and point at it.

Copy, or point.

Copied address

Every order stores the street

A move edits every order

Foreign key

The order stores a customer id

The street lives on the customer

Two orders, one customer

1. Customer 7 lives on Oak. Orders 1 and 2 store customer 7, not the street.

2. Change the customer to Elm. Both orders show Elm, because they join to the one row.

3. If the street had been copied, order 2 might still say Oak.

Problem 1. A column called id has two rows with the value 4. Can it be a primary key?

Answer. No. A primary key does not repeat. Add a different key, or stop inserting the duplicate.

Problem 2. An order names customer 99, and no customer 99 exists. What should the database do?

Answer. Reject the order. That is the foreign key. A table that accepts it will fail the join, or show an order with no customer, which is a fact you did not mean.

Lecture 2 · 50 minutes

SQL you can read

SELECT names the columns. FROM names the table. WHERE keeps rows. JOIN matches rows of two tables on a predicate, usually a foreign key. GROUP BY makes one result row per group, and every selected column is

grouped or aggregated.

NULL is not a value you can compare with equals. Use IS NULL. A condition that is unknown does not pass WHERE.

One query.

From the tables -> Join -> Filter -> Group, if you aggregate

Students and majors

1. Students: (1, Ada, CS), (2, Ben, math). Majors: (CS, engineering), (math, science).

2. Join on the major code. Ada's row gains engineering. Ben's row gains science.

3. WHERE school = engineering keeps Ada. Ben is gone, not deleted from the table. The query's result is a new table.

Problem 1. WHERE name = NULL returns no rows, and some names are NULL. What do you write instead?

Answer. WHERE name IS NULL. Equality with NULL is unknown, and unknown fails the filter.

Problem 2. You select student name and count of enrollments without GROUP BY. Why is that rejected?

Answer. Name is neither grouped nor aggregated. The count is one number and the name is many. Group by name, or drop the name.

Lecture 3 · 50 minutes

Algebra under the query

Selection keeps rows. Projection keeps columns. A join pairs rows that agree. The order of a filter and a join can change the cost and, if you are careless, the answer. A filter that mentions only one table can run before the join. Dropping a join column and then joining on it cannot.

SQL is a syntax for this algebra plus grouping. The graduate course optimizes the legal orders. You need to see an illegal one.

A filter that can move.

1. Start from students

2. Keep one major

3. Then join enrollments

4. The join is smaller

One illegal rewrite

1. You need the student's id to join to enrollments.

2. Projecting the student table down to the major alone throws the id away.

3. The later join has nothing to match. The rewrite is not equivalent. Cost does not matter if the answer changed.

Problem 1. When is it legal to filter before a join?

Answer. When the filter's columns all come from one side, and you are not using an outer join in a way that the filter would delete the unmatched rows you meant to keep. Inner join, one-sided predicate: filter early.

Problem 2. An outer join keeps students with no enrollment, and a later WHERE says course is not null. What remains?

Answer. Only students who have an enrollment. The outer join's extra rows have a null course and the filter removes them. The outer join was cancelled.

Lecture 4 · 50 minutes

A model of the entities

An entity is a thing you store: a student, a course. A relationship is a fact that connects them: enrolled. Cardinality says how many. A student enrolls in many courses. A course holds many students. That is many-to-many, and it needs its own table of pairs. A one-to-many can be a foreign key on the many side.

Draw this before you create tables. A table that mixes student and department phone will not survive an update.

Where the foreign key sits.

One department, many students

Student row holds department id

Many courses, many students

A third table holds student id and course id

Enrollment

1. Ada in 221 and 310 is two rows in the enrollment table, not two columns on Ada.

2. Adding a third course is an insert, not a schema change.

3. A design with course1, course2, course3 columns breaks on the fourth course and wastes space on the student who takes one.

Problem 1. A student has at most one advisor. Where do you store the advisor id?

Answer. On the student row, as a foreign key. A separate table is possible and unnecessary if the cardinality is really one.

Problem 2. Why is a list of course names stuffed into one text column a bad relationship?

Answer. You cannot put a foreign key on a slice of a string, you cannot index one course cleanly, and a rename of the course does not update the text. Use rows.

Lecture 5 · 50 minutes

Dependencies

A functional dependency X determines Y when two rows that agree on X agree on Y. It is a rule about legal tables, not an accident of today's data. A key determines every other column.

Third normal form, in the practical statement: for every dependency X determines A, either X is a key or A is part of a key. A table of student, advisor, and advisor phone violates it, because advisor determines phone and advisor is not a key of the student table. Split the phone onto the advisor.

One dependency, two tables.

Mixed

Student, advisor, phone

Two phones for one advisor if you miss an edit

Split

Student and advisor

Advisor and phone

A phone that changes

1. Kim advises Ada and Ben. Both rows say 555-0100.

2. Update only Ada's copy to 555-0199. The table now gives Kim two phones. The dependency advisor determines phone is broken.

3. Store the phone once, on Kim. Both students join to it.

Problem 1. The join of your two new tables has more rows than the table you started with. What failed?

Answer. The split was lossy. You dropped the column that tied the pieces together. A lossless split joins back to the same rows, no extras.

Problem 2. Today's data happens to satisfy a dependency you did not declare. Is the table in normal form for that dependency?

Answer. Normal form is about the dependencies you claim must always hold. An accident in the current rows is not a constraint. Declare it and enforce it, or do not pretend.

Lecture 6 · 50 minutes

An index, and a transaction

An index finds rows without reading the whole table. It helps the predicate it matches and slows every insert that must update it. Build it for the lookup you actually run.

A transaction is a group of writes that commit together or not at all. Transferring a seat from one student to another is one transaction. Two separate commits can leave the seat on both, or on neither, after a crash. CSCE

608 will open the log. This course asks you to name the transaction.

A transfer.

1. Begin

2. Remove the seat from A

3. Give the seat to B

4. Commit both

A lookup of one student

1. The table has a million enrollments. You want student 7.

2. No index: read every row.

3. An index on student id: a few reads in the index and the matching rows. The million-row scan was the missing index, not a slow disk in the abstract.

Problem 1. You index every column. What did you do to inserts?

Answer. You made each insert maintain every index. The rare lookup might be faster. The common insert is slower. Index the predicate you measured.

Problem 2. Why is CSCE 310 the prerequisite for CSCE 608, not a substitute?

Answer. 310 is the schema, the query, and the transaction boundary. 608 is pages, buffers, locks, and recovery. Taking both as two master's electives double-counts the subject. Take 310 first, then 608 inside the 36 credits.